

Casa di Cura Misericordia Navacchio S.r.l. — IT

Cruscotto Degenze

Documentazione tecnica e funzionale

Documento **Guida al programma, schema ER, architettura, installazione**

Versione **1.0 — 16/09/2026**

Riferimento **Prototipo v5 e Prompt ufficiale 1.0**

Applicazione **Python 3.12 · Flask · SQLAlchemy · ReportLab**

Rete **degenze-misnav · sql-misnav · cdcmisnav.local**

Redatto da **Micky — IT**

Indice

1. Che cos'è il Cruscotto Degenze	4
1.1 Da dove arriviamo	4
1.2 Le due versioni	4
2. Chi lo usa: ruoli e permessi	5
2.1 I cinque ruoli	5
2.2 Due regole speciali	5
3. Come si usa	6
3.1 Accesso	6
3.2 Home "Pazienti ricoverati"	6
3.3 Inserire un paziente	7
3.4 L'Agenda e la checklist	7
3.5 PDF del paziente	8
3.6 Eliminare un paziente	9
3.7 Gestione utenti e registro accessi	9
4. Come è fatto: l'architettura	10
4.1 Versione locale	10
4.2 Versione di rete	10
4.3 Cosa succede quando si scrive una nota	11
5. Il database: schema ER	12
5.1 Le relazioni in parole	14
5.2 Le tabelle	14
5.3 Chiavi e indici, con esempi	15
5.4 Eliminazione logica	16
5.5 L'istantanea dell'autore	16
5.6 Date e ore	16
6. Il codice: struttura e concetti Flask	17
6.1 Le cartelle del progetto	17
6.2 I concetti che incontrerai	17
7. Sicurezza e privacy	19
7.1 Password	19
7.2 Protezioni previste	19
7.3 Privacy e NIS2	19
8. Fase 1 — La versione locale	20
8.1 Cosa installare sul PC	20

8.2 Creare il progetto e avviarlo	20
8.3 Far provare il programma a medici e infermieri	20
8.4 Prova generale su una VM del tuo PC	21
9. Fase 2 — La versione di rete sulla VM	22
9.1 Cosa chiedere al consulente (Marco Bottalico)	22
9.2 Passi di installazione (sintesi di DEPLOY.md)	22
10. Backup, aggiornamenti e manutenzione	24
10.1 Backup	24
10.2 Aggiornare il programma	24
10.3 Controlli periodici	24
11. Piano di lavoro passo passo	25
12. Punti da confermare	25
13. Glossario	25

1. Che cos'è il Cruscotto Degenze

Il Cruscotto Degenze è un'applicazione web interna della Casa di Cura. Mostra in un'unica pagina i pazienti ricoverati nei tre reparti (**AOUP, Setting 1 — Low Care 1, Setting 2 — Low Care 2**) e offre a medici e infermieri tre strumenti per organizzare il ricovero:

- un'**Agenda** per ogni paziente, dove si scrivono note organizzative firmate;
- una **checklist di dimissione** di 20 voci;
- un **PDF** riepilogativo da stampare.

"Applicazione web" significa che **non si installa sui PC**: si apre con il browser (Edge o Chrome) a un indirizzo, come un sito. Il programma vero gira su un server; i PC di reparto mostrano solo le pagine.

Esempio concreto: l'infermiere Carlo Bianchi, dal PC dell'AOUP, apre <https://degenze.cdcmisnav.local>, entra con le sue credenziali e vede che Manetti Sergio (letto A-06) ha la dimissione presunta domani. Apre la sua Agenda, legge la nota del dott. Rossi, spunta "Familiari avvisati" nella checklist e aggiunge una nota. Un minuto dopo il dott. Rossi, da un altro PC, apre la stessa Agenda e trova la nota di Carlo in cima.

Cosa il Cruscotto NON è

Non è una cartella clinica e non sostituisce ResMedica. Registra solo dati organizzativi minimi: nome, cognome, posto letto, date di arrivo e di dimissione presunta, gravità, note e checklist. Niente diagnosi, terapie o codice fiscale. È una scelta di **minimizzazione dei dati** (GDPR): meno dati sanitari contiene, minori sono i rischi e gli obblighi.

1.1 Da dove arriviamo

- **Prototipo iniziale**: calendario Giorno 1...Giorno N e pulsanti "Note medico", "Note infermiere", "Dimetti", più un CSV per reparto.
- **Nuove richieste dei referenti**:
 - il calendario è sostituito dall'**Agenda**;
 - "Dimetti" è rimosso e resta "Elimina";
 - il CSV è sostituito dal **PDF per singolo paziente**.
- **Da CareFlow**: è stata presa la **checklist di dimissione**, inserita dentro l'Agenda.
- **Ultima richiesta**: in home servono **data di arrivo** e **dimissione presunta**, con i pazienti ordinati per dimissione.
- **Riferimento approvato**: il prototipo [PrototipoCruscotto_Degenze_v5.html](#) riassume tutto questo ed è la base per lo sviluppo.

1.2 Le due versioni

- **Versione locale** (Fase 1): tutto gira sul tuo PC e il database è un semplice file. Serve per sviluppare, provare e far vedere il programma a medici e infermieri.
- **Versione di rete** (Fase 2): lo stesso programma, installato su una macchina virtuale nel datacenter, con il database su SQL Server. Tutti i PC di reparto del dominio lo raggiungono all'indirizzo <https://degenze.cdcmisnav.local>.

2. Chi lo usa: ruoli e permessi

Ogni utente ha **un ruolo**, e il ruolo decide cosa può fare. Gli account li crea solo l'amministratore del programma, che poi consegna nome utente e password temporanea all'interessato. Al primo accesso ognuno sceglie la propria password.

2.1 I cinque ruoli

- **Amministratore** (ADMIN): può fare tutto.
 - Gestisce gli utenti (crea, disattiva, cambia ruolo, reimposta password, sblocca).
 - Consulta il registro accessi.
 - È l'unico che può eliminare un paziente.
 - Scrive note, spunta la checklist, cambia gravità e dimissione, inserisce pazienti, scarica PDF.
- **Medico** (MEDICO): scrive note, spunta la checklist, cambia gravità e dimissione presunta, inserisce pazienti, scarica PDF. Non elimina pazienti e non gestisce utenti.
- **Infermiere** (INFERMIERE): stessi permessi del medico.
- **OSS** (OSS): **solo lettura**. Vede home, Agenda e checklist, ma non scrive, non spunta e non scarica il PDF.
- **Amministrazione / IT** (AMMIN): **solo lettura più PDF**. Può scaricare e stampare la scheda del paziente.

2.2 Due regole speciali

- **Una nota la modifica o la elimina solo chi l'ha scritta.** Esempio: Carlo Bianchi vede "Modifica" ed "Elimina" solo sulle proprie note, non su quelle del dott. Rossi. Vale anche per l'amministratore: nessuno può cambiare le parole di un altro.
- **I permessi si controllano sul server, non solo sullo schermo.** Nascondere il pulsante "Elimina" a un infermiere non basta: un utente esperto potrebbe inviare la richiesta "a mano". Per questo ogni operazione viene ricontrollata dal programma prima di essere eseguita, e se il permesso manca la risposta è "403 — Accesso non consentito".

3. Come si usa

Cruscotto Degenze
Misericordia Navacchio

Pazienti

Pazienti ricoverati

Apri l'Agenda di un paziente per leggere e scrivere le note. Il pallino cambia la gravità.

● Critico ● Instabile ● Stabile ↻ Non valutato

AOUP 3 pazienti

Inserisci paziente

Paziente	Letto	Data arrivo	Dimissione presunta	Gravità	Azioni
Manetti Sergio	A-06	08/09/2026	17/09/2026	↻	Agenda 0 Scarica PDF
Rossi Mario	A-01	12/09/2026	19/09/2026	●	Agenda 5 Scarica PDF
Bianchi Anna	A-03	16/09/2026	23/09/2026	●	Agenda 0 Scarica PDF

Setting 1 — Low Care 1 2 pazienti

Inserisci paziente

Paziente	Letto	Data arrivo	Dimissione presunta	Gravità	Azioni
Conti Giulia	L1-02	04/09/2026	16/09/2026	●	Agenda 2 Scarica PDF
Neri Paolo	L1-05	14/09/2026	da definire	●	Agenda 0 Scarica PDF

Setting 2 — Low Care 2 1 paziente

Inserisci paziente

Paziente	Letto	Data arrivo	Dimissione presunta	Gravità	Azioni
Ferri Luca	L2-01	01/09/2026	21/09/2026	●	Agenda 0 Scarica PDF

Mario Rossi
Medico

Home page: i tre reparti con i pazienti ordinati per dimissione presunta (prototipo v5, utente Medico)

3.1 Accesso

- **Primo accesso:** l'utente inserisce nome utente e password temporanea, e il programma gli chiede subito di sceglierne una nuova (almeno 8 caratteri, con lettere e numeri).
- **Blocco dopo 5 tentativi:** dopo 5 password sbagliate di fila l'account resta bloccato per 15 minuti, oppure finché l'amministratore non lo sblocca. È una protezione contro chi prova a indovinare le password.
- **Uscita automatica:** dopo 15 minuti di inattività l'utente viene disconnesso. Nei reparti i PC sono condivisi, e così non resta aperta la sessione di qualcun altro.

3.2 Home "Pazienti ricoverati"

- **Riquadri:** uno per reparto, con il numero di pazienti.
- **Colonne:** Paziente, Letto, Data arrivo, Dimissione presunta, Gravità, Azioni.
- **Ordinamento:** in ogni reparto i pazienti sono ordinati per **dimissione presunta**, dalla più vicina alla più lontana.
 - Esempio in AOUP: Manetti (17/09), Rossi (19/09), Bianchi (23/09).
 - Chi non ha ancora una data va in fondo, con la scritta "da definire".
 - A parità di data decide il letto: A-02 prima di A-06.
- **Gravità:** il pallino colorato si cambia con un clic (non valutato → verde → giallo → rosso → non valutato), e il cambio è salvato subito.
- **Dimissione presunta:** si modifica cliccando sulla data.

- **Pulsanti:** **Agenda** (con il numero di note), **Scarica PDF**, e **Elimina** per il solo amministratore.

3.3 Inserire un paziente

Nuovo paziente
AOUP

Nome

Cognome

Posto letto

Data di arrivo

Data presunta di dimissione

Laura

Galli

A-01

09/16/2026

09/20/2026

Il letto A-01 risulta occupato da Rossi Mario.

Annulla

Inserisci paziente

Finestra di inserimento con il controllo del letto occupato

- **Campi obbligatori:** nome, cognome, posto letto e data di arrivo (già compilata con la data di oggi).
- **Campo facoltativo:** la data presunta di dimissione, perché all'arrivo spesso non si conosce.
- **Controlli automatici:**
 - la dimissione non può essere precedente all'arrivo;
 - il letto non può essere già occupato nello stesso reparto. Nell'esempio in figura, il letto A-01 risulta già occupato da Rossi Mario.

3.4 L'Agenda e la checklist

Agenda — Rossi Mario

AOUN, letto A-01, arrivo 12/09/2026, dimissione presunta 19/09/2026

▼ Checklist dimissione 5/20

☒ Paziente dimissibile	☒ Esami completati
☒ Referti acquisiti	☒ Terapia definita
☐ Prescrizioni predisposte	☐ Lettera di dimissione predisposta
☐ Lettera firmata	☐ Catetere: definito mantenimento/rimozione
☐ Medicazioni programmate	☐ Ausili prescritti
☐ Palliativisti attivati se necessari	☐ Continuità assistenziale attivata
☐ Data dimissione concordata	☒ Familiari avvisati
☐ Trasporto necessario definito	☐ Trasporto richiesto
☐ Trasporto confermato	☐ Documentazione consegnata
☐ Farmaci consegnati/prescritti	☐ Effetti personali consegnati

Scrivi come **Medico: Mario Rossi**, data 16/09/2026

Scrivi una nuova nota...

Scarica PDF

Chiudi

Agenda di Rossi Mario: checklist in alto, poi il campo per la nuova nota e lo storico

Checklist di dimissione

- Le 20 voci vanno da "Paziente dimissibile" a "Effetti personali consegnati".
- Il contatore (per esempio 5/20) diventa verde quando sono tutte spuntate.
- La spunta si salva al clic. Non si registra chi l'ha messa, come richiesto.
- Il riquadro si può chiudere per lasciare più spazio alle note.

Note

- **Ordine:** le più recenti sono in alto, raggruppate per giorno. Se due note hanno la stessa data, la più recente è quella inserita per ultima.
- **Firma:** ogni nota riporta il ruolo e il nome ("Medico: Mario Rossi") e la data.
- **Modifica:** se la nota è stata modificata compare "modificata il gg/mm/aaaa", e il testo precedente resta conservato nel database.
- **Eliminazione:** la nota sparisce dall'Agenda e dal PDF ma **non viene cancellata** dal database.

3.5 PDF del paziente

Il pulsante **Scarica PDF** produce un file A4 chiamato, per esempio, `Agenda_Rossi_Mario_2026-09-16.pdf`. Contiene:

- i dati del paziente;

- chi ha generato il documento e quando;
- la checklist con le caselle;
- lo storico completo delle note visibili;
- il numero di pagina.

Nella versione vera il PDF è creato **dal server**, quindi funziona anche su PC senza internet e senza librerie nel browser.

3.6 Eliminare un paziente

Solo l'amministratore, dopo una conferma. Il paziente sparisce dal cruscotto e il letto si libera, ma **i dati restano nel database**: stato "ELIMINATO", con chi l'ha eliminato e quando. Questa tecnica si chiama *eliminazione logica* (vedi capitolo 5.4).

3.7 Gestione utenti e registro accessi

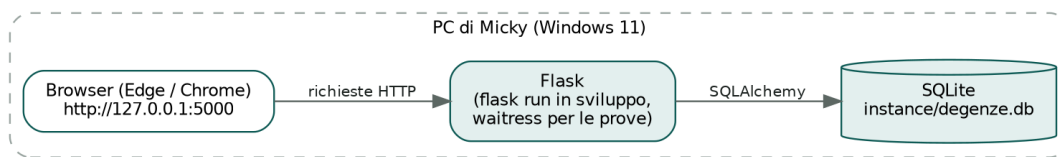
- **Gestione utenti**: l'amministratore crea gli account, li disattiva quando una persona lascia la struttura (non si cancellano mai, perché le sue note devono restare firmate), cambia ruolo, reimposta password e sblocca account.
- **Registro accessi**: elenca chi è entrato, chi ha creato o eliminato un paziente, chi ha scritto o modificato una nota, chi ha scaricato un PDF. Riporta sempre data, ora e indirizzo IP del PC, ma **mai il testo delle note**.

4. Come è fatto: l'architettura

Un'applicazione web è divisa in tre strati. Pensa a un ristorante:

- **Browser** = il cliente al tavolo: vede il menù (le pagine) e fa ordinazioni (clic, form).
- **Applicazione Flask** = la cucina: riceve l'ordinazione, controlla che si possa fare, prepara la risposta.
- **Database** = la dispensa: dove gli ingredienti (i dati) sono conservati in modo ordinato.

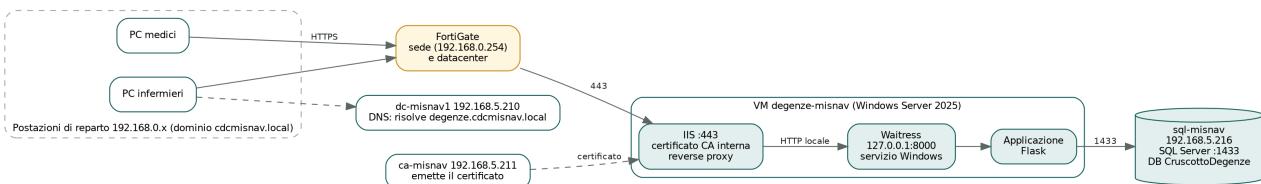
4.1 Versione locale



Fase 1: tutto sul PC di sviluppo

Sul tuo PC gira Flask, che risponde all'indirizzo `http://127.0.0.1:5000`. `127.0.0.1` significa "questo stesso computer": nessun altro PC lo raggiunge. Il database è il file `degenze.db` (SQLite), dentro la cartella `instance` del progetto. Per svilupparlo non serve installare nessun server di database.

4.2 Versione di rete



Fase 2: percorso di una richiesta da un PC di reparto al database

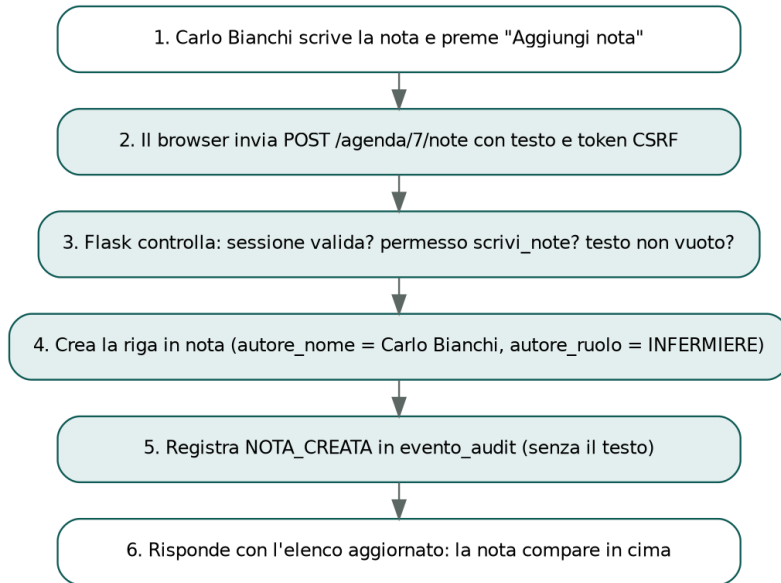
Seguiamo il percorso di una richiesta:

1. Il PC chiede al DNS di `dc-misnav1` (192.168.5.210) l'indirizzo di `degenze.cdcmisnav.local` e riceve l'IP della VM.
2. Il browser si collega in **HTTPS** (porta 443). Il traffico passa dai FortiGate, che devono consentirlo.
3. Sulla VM `degenze-misnav` risponde **IIS**. Presenta il certificato emesso dalla CA interna `ca-misnav`, che i PC del dominio riconoscono già come affidabile: niente avvisi di sicurezza.
4. IIS fa da **reverse proxy**: non esegue il programma, ma passa la richiesta a **Waitress**, che ascolta solo su `127.0.0.1:8000`, quindi è irraggiungibile dall'esterno della VM.
5. Waitress esegue l'**applicazione Flask**, che legge e scrive su **SQL Server** in `sql-misnav` (192.168.5.216, porta 1433).
6. La risposta torna indietro per la stessa strada.

Perché IIS davanti a Waitress?

Waitress è un ottimo server per applicazioni Python, ma non gestisce HTTPS. IIS invece lo gestisce bene, usa i certificati della vostra CA e lo conosci già. Così ognuno fa ciò che sa fare meglio: IIS cifra e smista, Waitress esegue Python.

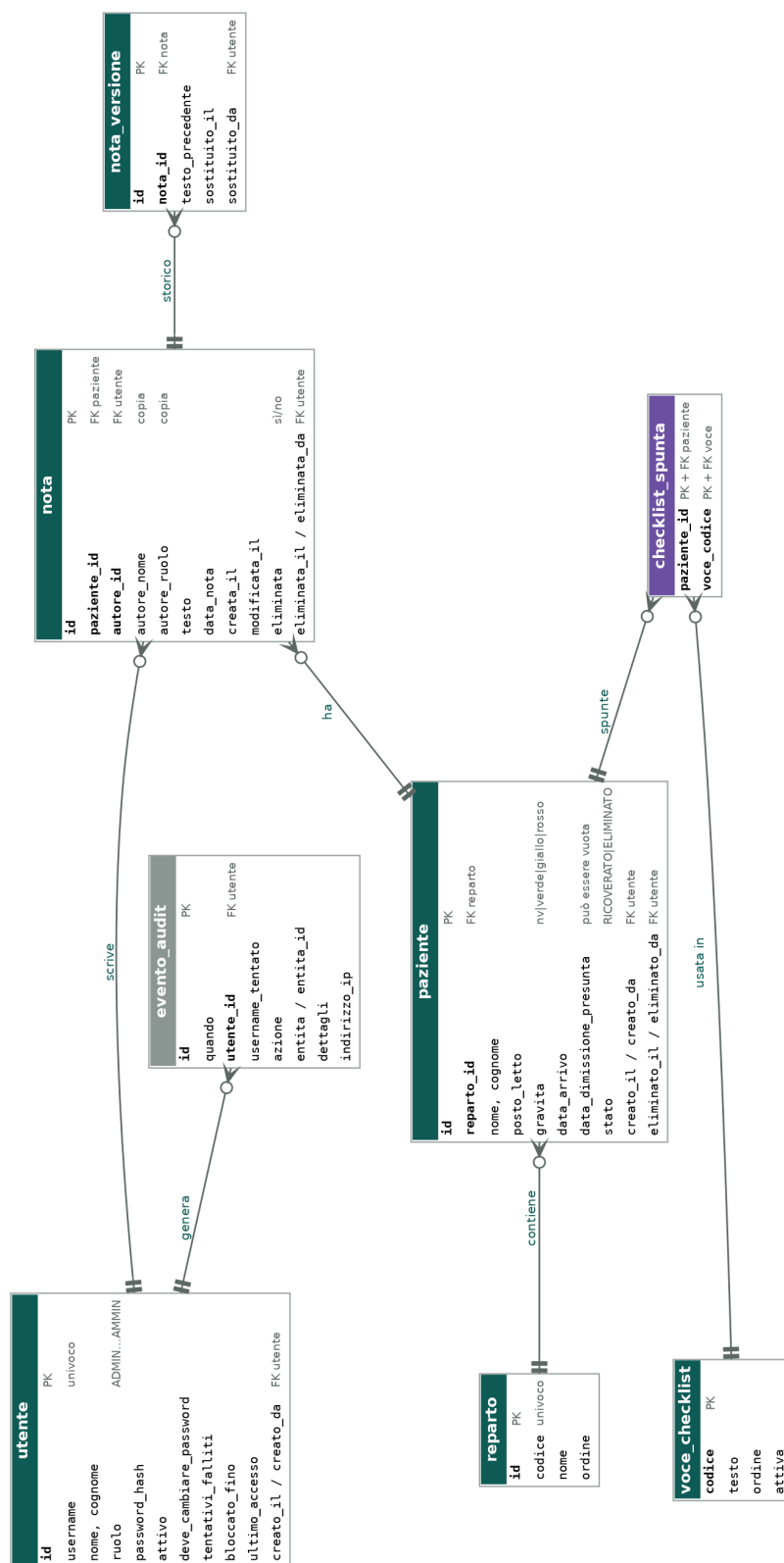
4.3 Cosa succede quando si scrive una nota



Flusso di una richiesta "Aggiungi nota"

5. Il database: schema ER

Uno schema **ER** (Entità-Relazioni) è la mappa del database. Le **entità** (i rettangoli) diventano tabelle, e le **relazioni** (le linee) dicono come le tabelle sono collegate.



Schema ER del Cruscotto Degenze (pagina ruotata). Le due stanghette indicano "uno", la zampa di gallina con il cerchio indica "zero o più".

5.1 Le relazioni in parole

- **reparto** → **paziente** (uno a molti): un reparto contiene molti pazienti, e ogni paziente sta in un solo reparto. Esempio: AOUP contiene Rossi, Bianchi e Manetti.
- **paziente** → **nota** (uno a molti): un paziente ha zero o più note.
- **utente** → **nota** (uno a molti): un utente scrive zero o più note, e ogni nota ha un solo autore.
- **nota** → **nota_versione** (uno a molti): ogni modifica di una nota salva il testo precedente come nuova versione.
- **paziente** ↔ **voce_checklist** (molti a molti): un paziente può avere più voci spuntate, e una voce può essere spuntata per più pazienti. Un legame molti-a-molti richiede sempre una **tabella ponte**, qui `checklist_spunta`: ogni sua riga significa "per questo paziente questa voce è spuntata".
- **utente** → **evento_audit** (uno a molti): ogni azione registrata ha il suo utente.
- **Altre relazioni con utente**, non disegnate per chiarezza: chi ha creato ed eliminato un paziente (`creato_da`, `eliminato_da`), chi ha eliminato una nota (`eliminata_da`), chi ha creato un utente (`creato_da`).

5.2 Le tabelle

reparto

I tre reparti, inseriti una volta sola dalla prima migrazione.

- `id`: numero identificativo (chiave primaria).
- `codice`: AOUP, S1, S2.
- `nome`: per esempio "Setting 1 — Low Care 1".
- `ordine`: posizione in home.

utente

- `id`: numero identificativo.
- `username`: nome utente, univoco (per esempio `cbianchi`).
- `nome`, `cognome`: dati anagrafici dell'utente.
- `ruolo`: uno dei cinque codici del capitolo 2.
- `password_hash`: **mai la password**, solo la sua "impronta" (vedi 7.1).
- `attivo`: vero o falso. Un utente disattivato non entra.
- `deve_cambiare_password`: vero dopo la creazione o dopo un reset.
- `tentativi_falliti`, `bloccato_fino`: gestiscono il blocco dopo 5 errori.
- `ultimo_accesso`, `creato_il`, `creato_da`: informazioni di controllo.

paziente

- `reparto_id`: **chiave esterna** verso reparto.
- `nome`, `cognome`, `posto_letto`: identificano il paziente nel reparto.
- `gravita`: `nv`, `verde`, `giallo` o `rosso`.
- `data_arrivo`: obbligatoria.
- `data_dimissione_presunta`: può essere vuota. Un vincolo impedisce che sia precedente all'arrivo.
- `stato`: `RICOVERATO` oppure `ELIMINATO`.
- `creato_il`, `creato_da`, `eliminato_il`, `eliminato_da`: chi ha fatto cosa e quando.

nota

- `paziente_id`, `autore_id`: chiavi esterne verso paziente e utente.
- `autore_nome`, `autore_ruolo`: **copia** di nome e ruolo al momento della scrittura (vedi 5.5).
- `testo`: fino a 2000 caratteri.
- `data_nota`: il giorno mostrato.
- `creata_il`: data e ora precise, usate internamente.
- `modificata_il`: vuota finché la nota non viene modificata.
- `eliminata`, `eliminata_il`, `eliminata_da`: eliminazione logica.

nota_versione

Una riga per ogni modifica.

- `nota_id`: la nota modificata.
- `testo_precedente`: il testo prima della modifica.
- `sostituito_il`, `sostituito_da`: quando e chi ha fatto la modifica.

Esempio: il dott. Rossi scrive "Miglioramento clinico." e poi lo corregge in "Miglioramento clinico. Dimissione tra 3 giorni.". In `nota` resta il testo nuovo; in `nota_versione` finisce "Miglioramento clinico."

voce_checklist e checklist_spunta

- **voce_checklist**: le 20 voci, ognuna con un **codice fisso** (per esempio `LETTERA_FIRMA`), un testo e un ordine.
- **checklist_spunta**: contiene solo le coppie (paziente, codice) spuntate. Togliere la spunta significa cancellare la riga: è l'unico caso di cancellazione fisica previsto, perché una spunta non è un dato da conservare.

Esempio: se per Rossi Mario (id 1) sono spuntate "Paziente dimissibile" e "Familiari avvisati", la tabella contiene due righe: (1, `DIMISSIBILE`) e (1, `FAMILIARI`).

evento_audit

Il registro accessi.

- `quando`: data e ora dell'evento.
- `utente_id`: l'utente, oppure vuoto se il login è fallito con un nome inesistente.
- `username_tentato`: il nome usato nei login falliti.
- `azione`: per esempio `NOTA_CREATA`.
- `entita`, `entita_id`: per esempio `nota` e 57.
- `dettagli`: un breve testo, mai il contenuto delle note.
- `indirizzo_ip`: l'IP del PC da cui è partita la richiesta.

5.3 Chiavi e indici, con esempi

- **Chiave primaria (PK)**: il "codice fiscale" della riga, unico e mai ripetuto. Esempio: il paziente con `id = 7`.
- **Chiave esterna (FK)**: un campo che punta alla PK di un'altra tabella. Esempio: una nota con `paziente_id = 7` appartiene al paziente 7. Il database rifiuta una nota con `paziente_id = 999` se quel paziente non esiste.
- **Indice**: come l'indice analitico di un libro, rende veloci le ricerche. L'indice su (`stato`, `reparto_id`, `data_dimissione_presunta`) serve proprio alla home: "dammi i ricoverati di AOUP

ordinati per dimissione".

- **Indice univoco filtrato:** (`reparto_id`, `posto_letto`) è univoco **solo tra i pazienti ricoverati**. Così il letto A-01 non può avere due ricoverati insieme, ma può comparire in tanti pazienti eliminati in passato. SQLite e SQL Server lo supportano entrambi.

5.4 Eliminazione logica

Invece di `DELETE` (cancellare la riga) si usa un `UPDATE` che la marca come eliminata. Esempio: eliminare Rossi Mario imposta `stato = 'ELIMINATO'`, `eliminato_da = 1`, `eliminato_il = 2026-09-16 10:32`. La home mostra solo i `RICOVERATO`, quindi Rossi sparisce dalla vista, ma le sue note e la sua checklist restano consultabili dal database in caso di contestazioni o verifiche.

5.5 L'istantanea dell'autore

Nella nota si salvano sia `autore_id` sia una **copia** di nome e ruolo. Esempio: Carlo Bianchi scrive una nota oggi come infermiere; l'anno prossimo diventa coordinatore e l'amministratore gli cambia ruolo. Le vecchie note devono continuare a dire "Infermiere: Carlo Bianchi", perché quello era il suo ruolo quando le ha scritte. `autore_id` serve invece per decidere chi può modificarle.

5.6 Date e ore

Nel database date e ore si salvano in **UTC**, l'ora universale, e si mostrano in ora italiana. Così il cambio tra ora legale e solare non crea note "nel futuro" o doppie. `data_nota` è invece già la data italiana del giorno, perché è quella che si vede.

6. Il codice: struttura e concetti Flask

6.1 Le cartelle del progetto

CruscottoDegenze/	
app/	
__init__.py	crea l'applicazione (create_app)
config.py	impostazioni per sviluppo, test, produzione
estensioni.py	database, login, protezione CSRF
models.py	le tabelle del capitolo 5 scritte in Python
permessi.py	chi può fare cosa (capitolo 2)
audit.py	scrive nel registro accessi
comandi.py	comandi da terminale: crea-admin, dati-demo
auth/	accesso, uscita, cambio password
pazienti/	home, inserimento, gravità, dimissione, eliminazione
agenda/	note e checklist
pdf/	generazione del PDF
utenti/	gestione utenti e registro accessi
templates/	le pagine HTML
static/	CSS, JavaScript, font
migrations/	storia delle modifiche al database
tests/	test automatici
deploy/	script e guida per la VM
.env	impostazioni segrete (NON va su Git)
requirements.txt	elenco delle librerie con le versioni
wsgi.py	punto di avvio per Waitress

6.2 I concetti che incontrerai

- **Route**: collega un indirizzo a una funzione Python. Esempio: `@bp.route("/")` associato alla funzione `home()` fa sì che aprendo `/` Flask esegua `home()`, che legge i pazienti e restituisce la pagina.
- **Template (Jinja2)**: una pagina HTML con dei "buchi" riempiti dai dati. Esempio: `{{ paziente.cognome }}` diventa "Rossi". Jinja fa anche l'*escape* automatico: se qualcuno scrive `<script>` in una nota, viene mostrato come testo e non eseguito.
- **Blueprint**: un gruppo di route su un tema. `pazienti`, `agenda` e `utenti` sono blueprint separati, così il codice resta ordinato.
- **Application factory**: la funzione `create_app()` costruisce l'applicazione con la configurazione giusta. Esempio: nei test usa un database temporaneo, in produzione SQL Server.
- **ORM (SQLAlchemy)**: permette di lavorare con classi Python invece di scrivere SQL. `Paziente.query.filter_by(stato="RICOVERATO")` diventa automaticamente `SELECT ... WHERE stato = 'RICOVERATO'`, nella variante giusta per SQLite o SQL Server.
- **Migrazione (Flask-Migrate)**: un file che descrive una modifica al database. Esempio: aggiungere la colonna `data_dimissione_presunta` genera una migrazione. Il comando `flask db upgrade` la applica sia al tuo SQLite sia, più avanti, a SQL Server. Non si modificano mai le tabelle "a mano".
- **Session (Flask-Login)**: dopo il login il browser riceve un cookie firmato che dice "questo è l'utente 3". A ogni clic Flask lo legge e sa chi sei.
- **Ambiente virtuale (venv)**: una cartella `.venv` con le librerie di questo solo progetto, così non si mescolano con altri programmi Python del PC.
- **File .env**: contiene le impostazioni che cambiano tra PC e server e i segreti, per esempio la `SECRET_KEY` o la password del database. Non va mai su Git; al suo posto si versiona `.env.example`, senza valori reali.

- **Test automatici (pytest):** piccoli programmi che verificano le regole. Esempio: "un OSS che prova a eliminare un paziente deve ricevere 403". Si lanciano con `pytest` prima di ogni rilascio.
- **Git:** tiene la storia del codice. Dopo ogni passo riuscito si fa un *commit*, così se il passo successivo rompe qualcosa si torna indietro in un attimo.

7. Sicurezza e privacy

7.1 Password

Le password non vengono mai salvate. Si salva il loro **hash**: una trasformazione a senso unico che dalla password produce una stringa lunga, e da cui non si può tornare indietro. Al login il programma ricalcola l'hash della password digitata e lo confronta con quello salvato. Esempio: se qualcuno copiasse il database, troverebbe qualcosa come `bcrypt:32768:8:1$...` e non "Infermiere2026".

7.2 Protezioni previste

- **HTTPS** con il certificato della CA interna: le password non viaggiano in chiaro sulla LAN.
- **CSRF**: ogni form contiene un codice segreto. Una pagina esterna non può quindi far compiere azioni a un utente collegato a sua insaputa.
- **Controllo dei permessi sul server** per ogni operazione.
- **Blocco** dopo 5 tentativi e **timeout** dopo 15 minuti di inattività.
- **Cookie** di sessione protetti (`HttpOnly`, `Secure`, `SameSite`).
- **Escape** automatico dei testi e **query solo tramite ORM**, contro gli attacchi di tipo XSS e SQL injection.
- **Nessuna risorsa esterna** (niente CDN): il programma funziona anche se internet non c'è.
- **Account SQL con permessi minimi**: l'applicazione può leggere e scrivere ma non cancellare pazienti o note, né modificare il registro accessi.
- **Waitress raggiungibile solo dalla VM stessa**; sulla VM sono aperte solo le porte 80 e 443.
- **SentinelOne** sulla VM, come sugli altri server.

7.3 Privacy e NIS2

- **Minimizzazione**: solo i dati indispensabili per organizzare il ricovero. Nelle note si scrivono informazioni organizzative; quelle cliniche restano in ResMedica.
- **Tracciabilità**: il registro accessi permette di rispondere a "chi ha visto o fatto cosa", come richiedono GDPR e NIS2.
- **Conservazione**: nulla si cancella fisicamente. Per quanto tempo conservare pazienti eliminati e registro accessi va deciso con la Direzione e il DPO.
- **Adempimenti organizzativi**: prima dell'avvio va informato il DPO, per esempio per aggiornare il registro dei trattamenti. Non è un tema tecnico, ma fa parte del progetto.

8. Fase 1 — La versione locale

8.1 Cosa installare sul PC

```
winget install Python.Python.3.12
winget install Git.Git
winget install Microsoft.VisualStudioCode
```

Dopo l'installazione chiudi e riapri PowerShell, poi verifica le versioni con `py --version` e `git --version`. Se winget non è disponibile, scarica gli installer dai siti ufficiali (python.org, git-scm.com, code.visualstudio.com). Durante l'installazione di Python spunta "Add python.exe to PATH".

8.2 Creare il progetto e avviarlo

Questi comandi li userai quando l'assistente AI avrà creato i file del passo 1. Servono a capire la sequenza.

```
cd C:\Progetti\CruscottoDegenze
py -3.12 -m venv .venv                # crea l'ambiente virtuale
.\venv\Scripts\Activate.ps1          # lo attiva: il prompt mostra (.venv)
pip install -r requirements.txt       # installa le librerie
copy .env.example .env               # crea il file delle impostazioni
# genera una chiave casuale e copiala in SECRET_KEY dentro .env:
python -c "import secrets; print(secrets.token_hex(32))"
flask db upgrade                      # crea il database SQLite
flask crea-admin                     # crea il tuo account amministratore
flask dati-demo                      # (solo sviluppo) pazienti e utenti di prova
flask run                            # avvia: apri http://127.0.0.1:5000
```

Se PowerShell blocca Activate.ps1

Esegui una volta `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`. Se una GPO aziendale lo impedisce, usa il Prompt dei comandi con `.venv\Scripts\activate.bat`.

Esempio del file `.env` in sviluppo:

```
FLASK_APP=wsgi.py
FLASK_DEBUG=1
SECRET_KEY=incolla-qui-la-stringa-generata
DATABASE_URL=sqlite:///degenze.db
MINUTI_INATTIVITA=15
```

8.3 Far provare il programma a medici e infermieri

- **Sul tuo PC:** la soluzione più semplice e sicura è far provare il programma sulla tua postazione, con i dati demo.
- **Da altri PC:** per farlo provare da un'altra postazione dovresti avviarlo con `waitress-serve --listen=0.0.0.0:8000 --call app:create_app` e aprire la porta nel firewall. La connessione però non è cifrata, quindi fallo **solo con dati finti** e dopo averlo concordato con il consulente. In alternativa aspetta la VM di prova (8.4).
- **Raccolta dei commenti:** annota ogni osservazione in un file `feedback.md`, con chi l'ha fatta e la data. Poi decidi con i referenti cosa entra in questa versione e cosa dopo.

8.4 Prova generale su una VM del tuo PC

Prima di toccare il datacenter conviene ripetere l'installazione su una macchina virtuale gratuita:

- **Hyper-V:** si attiva su Windows 11 Pro con `Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All` da PowerShell come amministratore, e richiede un riavvio.
- **Sistema operativo:** Windows Server 2025 *Evaluation*, gratuito per 180 giorni, dal Microsoft Evaluation Center.
- **Database:** SQL Server Express, gratuito, installato sulla stessa VM di prova.
- **Risorse:** bastano 2 vCPU, 4 GB di RAM e 60 GB di disco. Sul PC servono almeno 16 GB di RAM in totale.
- **Certificato:** in prova si usa un certificato autofirmato. Il browser mostrerà un avviso ed è normale.

Se la prova generale va liscia seguendo `DEPLOY.md`, l'installazione vera sarà una ripetizione.

9. Fase 2 — La versione di rete sulla VM

9.1 Cosa chiedere al consulente (Marco Bottalico)

- **VM** `degenze-misnav`: Windows Server 2025 Standard, 2 vCPU, 4 GB di RAM, disco da 80 GB, nel dominio `cdcmisnav.local`, con IP statico nella rete 192.168.5.0/24 (per esempio il primo libero dopo .217) e agente SentinelOne.
- **SQL Server** su `sql-misnav`:
 - conferma che si può ospitare il database `CruscottoDegenze` sull'istanza esistente, e il nome esatto dell'istanza;
 - conferma che è attiva l'autenticazione mista (Windows + SQL);
 - verifica che il database rientri nel piano di backup.
- **Regole FortiGate** (sede e datacenter):
 - postazioni 192.168.0.0/24 → VM sulla porta 443 (e 80 per il reindirizzamento);
 - VM → `sql-misnav` sulla porta 1433.
- **Certificato**: il modello "Server Web" deve essere pubblicato sulla CA `ca-misnav`, con permesso di richiesta per la VM.

9.2 Passi di installazione (sintesi di DEPLOY.md)

1. Record DNS su `dc-misnav1`:

```
Add-DnsServerResourceRecordA -ZoneName "cdcmisnav.local" -Name "degenze" `
                              -IPv4Address <IP della VM>
```

2. Database e login su `sql-misnav`, con lo script `crea_database.sql`. L'idea, in breve:

```
CREATE DATABASE CruscottoDegenze;
CREATE LOGIN degenze_app WITH PASSWORD = '...'; -- usato dal programma
CREATE LOGIN degenze_migra WITH PASSWORD = '...'; -- usato solo per le migrazioni
USE CruscottoDegenze;
CREATE USER degenze_app FOR LOGIN degenze_app;
ALTER ROLE db_datareader ADD MEMBER degenze_app;
ALTER ROLE db_datawriter ADD MEMBER degenze_app;
CREATE USER degenze_migra FOR LOGIN degenze_migra;
ALTER ROLE db_owner ADD MEMBER degenze_migra;
-- dopo la prima migrazione, "niente cancellazioni" garantito anche dal database:
DENY DELETE ON dbo.paziente TO degenze_app;
DENY DELETE ON dbo.nota TO degenze_app;
DENY DELETE ON dbo.nota_versione TO degenze_app;
DENY DELETE ON dbo.utente TO degenze_app;
DENY UPDATE, DELETE ON dbo.evento_audit TO degenze_app;
```

3. Prerequisiti sulla VM:

- Python 3.12 per tutti gli utenti;
 - *ODBC Driver 18 for SQL Server*;
 - ruolo IIS, con `Install-WindowsFeature Web-Server -IncludeManagementTools`;
 - moduli IIS *URL Rewrite* e *Application Request Routing* (scaricati da Microsoft);
 - NSSM, per trasformare Waitress in servizio.
4. **Applicazione**: copia in `C:\Apps\CruscottoDegenze`, poi esegui `deploy\installa.ps1`. Lo script crea l'ambiente virtuale, installa le librerie, prepara `.env`, lancia le migrazioni con `degenze_migra` e registra il servizio `CruscottoDegenze`.

5. **Certificato:** da `certlm.msc` sulla VM, *Personale* → *Richiedi nuovo certificato*, modello "Server Web", con nome DNS `degenze.cdcmisnav.local` nel campo **SAN**.
6. **IIS:**
 - sito sulla porta 443 con il certificato;
 - `web.config` di reindirizzamento verso `http://127.0.0.1:8000`;
 - proxy abilitato in *Application Request Routing* → *Server Proxy Settings*;
 - regola che porta la porta 80 su HTTPS.
7. **Firewall di Windows:** consentire in ingresso solo le porte 80 e 443.
8. **Verifica:** da un PC di reparto apri `https://degenze.cdcmisnav.local`. Deve comparire il lucchetto senza avvisi, e login, note, checklist e PDF devono funzionare. Riavvia la VM e controlla che il servizio riparta da solo.
9. **Utenti reali:** crea con `flask crea-admin` il primo amministratore, poi dal programma gli account di medici e infermieri, e consegna le credenziali.

Esempio del file `.env` in produzione (i valori veri li sceglierai tu):

```
FLASK_APP=wsgi.py
FLASK_DEBUG=0
SECRET_KEY=<nuova stringa diversa da quella di sviluppo>
# nel file vero DATABASE_URL sta su UNA sola riga (qui è spezzata per motivi di spazio)
DATABASE_URL=mssql+pyodbc://degenze_app:<password>@sql-misnav.cdcmisnav.local/CruscottoDegenze
?driver=ODBC+Driver+18+for+SQL+Server&Encrypt=yes
MINUTI_INATTIVITA=15
COOKIE_SICURI=1
```

Due punti da verificare con il consulente

- **Certificato di SQL Server:** se non è emesso dalla CA interna, la connessione con `Encrypt=yes` può fallire. In quel caso si aggiunge temporaneamente `TrustServerCertificate=yes` e poi si sistema il certificato.
- **PC ancora nel vecchio dominio** `misericordianavacchio.local`: potrebbero non risolvere `cdcmisnav.local` e non fidarsi della CA `ca-misnav`. Servono un inoltro condizionale DNS e la distribuzione del certificato della CA via GPO, come avete fatto per la catena del FortiGate.

10. Backup, aggiornamenti e manutenzione

10.1 Backup

- **Database:** backup completo giornaliero di `CruscottoDegenze` su `sql-misnav`, con conservazione da concordare (per esempio 30 giorni). È la cosa più importante, perché il programma si reinstalla in mezz'ora, i dati no.
- **VM:** snapshot o backup della VM prima di ogni aggiornamento.
- **File** `.env`: copia in un luogo sicuro con accesso limitato, perché contiene le password.
- **Prova di ripristino:** un backup mai provato non è un backup. Almeno una volta, ripristina il database su SQL Server Express nella VM di prova e verifica che il programma lo legga.

10.2 Aggiornare il programma

Con `deploy\aggiorna.ps1`, fuori dall'orario di punta:

1. backup del database;
2. arresto del servizio `CruscottoDegenze`;
3. aggiornamento del codice (per esempio `git pull` o copia della nuova versione);
4. `pip install -r requirements.txt`;
5. `flask db upgrade`, con il login delle migrazioni;
6. riavvio del servizio e verifica.

Se qualcosa non va: si ferma il servizio, si ripristinano codice e database salvati e si riavvia.

10.3 Controlli periodici

- **Log:** i log applicativi si trovano in `C:\Apps\CruscottoDegenze\logs`. Controlla gli errori, cercando le righe con `ERROR`.
- **Utenti:** disattiva subito gli account di chi lascia la struttura. Una volta al mese verifica gli account attivi e gli ultimi accessi.
- **Aggiornamenti:** aggiorna Windows sulla VM (GPO `Server_Download_Updates`) e, periodicamente, le librerie Python, sempre passando prima dalla VM di prova.

11. Piano di lavoro passo passo

Ogni tappa si chiude solo quando il risultato è verificato. A ogni passo riuscito si fa un commit Git.

1. **Tappa 0 — Chiusura dei requisiti.** Far approvare il prototipo v5 ai referenti di medici e infermieri e sciogliere i punti del capitolo 12.
2. **Tappa 1 — Preparazione del PC.** Installare Python, Git e VS Code (capitolo 8.1) e scegliere l'assistente AI. Creare la cartella `C:\Progetti\CruscottoDegenze` con dentro il prototipo v5 e il prompt.
3. **Tappa 2 — Avvio con l'assistente AI.** Incollare il prompt e allegare il prototipo. Leggere con me la risposta, i dubbi e la proposta del passo 1, prima di dare il via.
4. **Tappa 3 — Fase 1, passi 1-10.** Un passo alla volta: l'assistente scrive, tu esegui, verifichi e mi porti eventuali errori o dubbi. Commit a ogni passo.
5. **Tappa 4 — Prove con gli utenti.** Sessioni con medici e infermieri sulla versione locale, raccolta dei commenti in `feedback.md` e correzioni.
6. **Tappa 5 — Prova generale.** Installazione completa sulla VM Hyper-V seguendo `DEPLOY.md`, prova di backup e ripristino, correzione della guida dove non era chiara.
7. **Tappa 6 — Richieste al consulente.** VM, IP, DNS, database e login SQL, regole FortiGate, certificato, backup (capitolo 9.1).
8. **Tappa 7 — Installazione in produzione.** Passi del capitolo 9.2, creazione degli utenti reali, breve formazione nei reparti, avvio.
9. **Tappa 8 — Prime settimane.** Controllo dei log e dei commenti, piccoli aggiornamenti seguendo il capitolo 10.2.

12. Punti da confermare

1. **Modifica della dimissione presunta:** è inclusa nel prompt (clic sulla data in home, per amministratore, medici e infermieri), perché senza di essa la data resterebbe fissa. Confermare con i referenti.
2. **Timeout di inattività:** 15 minuti vanno bene per i reparti o serve un valore diverso?
3. **Registro accessi:** confermare gli eventi tracciati e per quanto tempo conservarli.
4. **Database:** SQL Server su `sql-misnav` va verificato con il consulente (istanza, autenticazione mista, backup). L'alternativa è PostgreSQL sulla VM dell'applicazione, con modifiche solo alla configurazione.
5. **Nome e IP:** confermare l'indirizzo `degenze.cdcmisnav.local` e l'IP della VM.
6. **Postazioni nel vecchio dominio:** quali PC di reparto non sono ancora migrati e come raggiungeranno il programma.
7. **Amministratori del programma:** solo l'IT, o anche un referente per reparto?
8. **Conservazione e privacy:** parere del DPO sui tempi di conservazione dei pazienti eliminati.

13. Glossario

- **ARR / URL Rewrite:** moduli di IIS che permettono di inoltrare le richieste a un altro programma (reverse proxy).
- **CA interna:** l'autorità di certificazione del dominio (`ca-misnav`), che firma i certificati di cui i PC del dominio si fidano.
- **CSRF:** attacco in cui una pagina esterna fa compiere azioni a un utente collegato. Si previene con un codice segreto in ogni form.
- **Eliminazione logica:** marcare una riga come eliminata invece di cancellarla.

- **FK / PK:** chiave esterna e chiave primaria (capitolo 5.3).
- **Hash:** impronta a senso unico di un dato, usata per le password.
- **Migrazione:** file che descrive una modifica alla struttura del database.
- **NSSM:** piccolo programma che trasforma un eseguibile qualsiasi in un servizio Windows.
- **ORM:** libreria che traduce classi Python in tabelle e query SQL.
- **Reverse proxy:** server che riceve le richieste dai client e le passa a un altro server interno.
- **SAN:** campo del certificato con i nomi DNS validi. I browser moderni guardano solo questo.
- **UTC:** ora universale, senza ora legale.
- **venv:** ambiente virtuale Python con le librerie del solo progetto.
- **Waitress:** server applicativo Python adatto a Windows.